UNIT 3 · TOPIC 3.2

Protecting Networks: Managerial Controls and Wireless Security

Assignment

Instructions: 18 practice questions + 3 coding assignment(s), 31 points total. Show your work for short-answer items. Coding assignments may be completed here or in the interactive auto-graded environment on the course site (same problem, same points).

Name: **Date:** **Class:**

Practice Questions

3.2.a · Wi-Fi basics and wireless security protocols (WPA2/WPA3)

1. Which cipher suite does WPA2 use to provide both confidentiality and data integrity? (1 pt)

- ○ A. RC4-TKIP
- ○ B. AES-CCMP
- ○ C. DES-CBC
- ○ D. 3DES-ECB

2. What is the primary security advantage of WPA3-SAE over WPA2-PSK? (1 pt)

- ○ A. WPA3-SAE prevents offline dictionary attacks against captured handshakes
- ○ B. WPA3 uses a longer password by default
- ○ C. WPA3 does not require any password
- ○ D. WPA3 uses RC4, which is faster

3. A student says 'I hid my SSID so attackers cannot find my network.' What is wrong with this reasoning? (1 pt)

- ○ A. Nothing is wrong; hidden SSIDs are invisible to all scanners
- ○ B. SSID hiding increases encryption strength
- ○ C. Hidden SSIDs still appear in probe responses and any passive scanner reveals them
- ○ D. SSID hiding is only available on WPA3

4. Select ALL configuration choices that represent best practice for a school Wi-Fi network. (3 pts)

- □ A. Use WPA2-AES or WPA3 for the student SSID
- □ B. Use WEP for legacy device compatibility
- □ C. Isolate guest devices on a separate VLAN
- □ D. Keep AP firmware updated to patch protocol vulnerabilities

(Select ALL that apply.)

5. Which WPA2 mode requires individual credentials via a RADIUS server and is preferred for corporate environments? (1 pt)

- ○ A. WPA2-PSK (Personal)
- ○ B. WPA2-OWE
- ○ C. WPA2-TKIP
- ○ D. WPA2-EAP (Enterprise)

6. Name the WPA3 property that ensures compromising today's session key cannot decrypt previously captured sessions. (1 pt)

3.2.b · Managerial controls and security policies

7. What distinguishes a managerial (administrative) control from a technical control? (1 pt)

- ○ A. Managerial controls are policies/procedures enforced through training and process, not automatically by hardware or software
- ○ B. Managerial controls are always more expensive
- ○ C. Managerial controls only apply to physical security
- ○ D. There is no meaningful difference

8. An organization requires new employees to sign a document defining permitted and prohibited use of company systems. What is this document called? (1 pt)

- ○ A. Firewall configuration
- ○ B. Acceptable Use Policy (AUP)
- ○ C. Access control list
- ○ D. Badge access log

9. Which of the following are examples of managerial controls? Select ALL that apply. (2 pts)

- □ A. A documented least-privilege access review process
- □ B. A locked server room door
- □ C. Annual security awareness training
- □ D. A firewall rule blocking a malicious IP

(Select ALL that apply.)

10. Why is 'privilege creep' a problem that a least-privilege policy specifically addresses? (1 pt)

- ○ A. It refers to accounts gradually accumulating unnecessary access rights over time without review
- ○ B. It refers to passwords getting weaker over time
- ○ C. It only affects physical badge access, not digital accounts
- ○ D. It is a technical vulnerability in firewalls

11. Name the recurring managerial control that teaches employees to recognize social engineering and follow security policy, often including simulated phishing emails. (1 pt)

12. A badge reader that requires a PIN and physical badge to open a data center door, is this a technical or physical control? (1 pt)

- ○ A. Physical, it restricts physical access to a space, regardless of using electronic components
- ○ B. Technical, anything involving electronics is automatically technical
- ○ C. Managerial, it requires an employee to remember a PIN
- ○ D. None of the above

3.2.c · Layered network defense overview

13. What is the core justification for defense in depth (layered defense)? (1 pt)

- ○ A. It is required by law in every jurisdiction
- ○ B. It reduces the total number of controls an organization needs
- ○ C. Every individual control has weaknesses or can be misconfigured, so no single control should be relied on alone
- ○ D. It guarantees an attacker can never succeed

14. Endpoint antivirus/EDR software installed on individual employee laptops belongs to which defense-in-depth layer? (1 pt)

- ○ A. Perimeter
- ○ B. Human
- ○ C. Data
- ○ D. Host

15. Which of the following are examples of network-layer controls (as distinct from perimeter or host)? Select ALL that apply. (2 pts)

- □ A. VLAN segmentation separating departments
- □ B. WPA3 wireless security protocol
- □ C. A border firewall at the internet gateway
- □ D. Full-disk encryption on a laptop

(Select ALL that apply.)

16. Why does the human/managerial layer 'span' all the other layers rather than sitting at one specific point? (1 pt)

- ○ A. It only applies to physical security
- ○ B. It only matters during an active breach
- ○ C. It replaces the need for any technical controls
- ○ D. It keeps every technical layer correctly configured, monitored, and improved, and catches what automated controls miss

17. Name the layer that protects data itself (e.g., via encryption) regardless of which device or network segment it is currently on. (1 pt)

18. An organization deploys only a single, very strong perimeter firewall and no other controls. What is the main risk of this approach? (1 pt)

- ○ A. None, a strong enough single control is always sufficient
- ○ B. Firewalls are technically incapable of stopping any attacks
- ○ C. Different attack vectors (e.g., ARP spoofing on the internal network, phishing) bypass the perimeter entirely, so a single control leaves other layers completely undefended
- ○ D. It costs more than layered defenses

Coding Assignments (Python 3)

3.2.a · Wi-Fi basics and wireless security protocols (WPA2/WPA3)

3 pts · auto-graded on the site

Coding problem, 3.2.a: Wi-Fi configuration security evaluator

Skill: Mitigate Risk · **Maps to:** 3.2.A · **Language:** Python 3 (pytest)

Problem

You are building an automated Wi-Fi configuration checker for Crestfield Analytics, mirroring what enterprise wireless assessment tools do. Given a dictionary describing one access point's configuration, return the sorted list of security gap codes that configuration has, based on the criteria from the lesson (protocol strength, PSK password length, guest network isolation, admin interface password strength, and firmware currency).

Function contract

Implement `evaluate_wifi_config(cfg)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `cfg`: `dict` with: - `"protocol"`: `str`, one of `"WPA3"`, `"WPA2-AES"`, `"WPA2-TKIP"`, `"WEP"`, `"OPEN"` - `"mode"`: `str`, `"personal"` or `"enterprise"` - `"psk_length"`: `int`, length of the pre-shared key/password (only meaningful when `mode == "personal"`) - `"guest_network_isolated"`: `bool`, whether guest devices are on a separate, isolated SSID/VLAN - `"admin_password_strong"`: `bool`, whether the AP's management interface has a strong unique password - `"firmware_current"`: `bool`, whether the AP is running current (patched) firmware

Output

A sorted `list[str]` of gap codes present in the configuration (empty list if there are no gaps). Possible gap codes:

- `"insecure_protocol"`, protocol is `"WEP"`, `"WPA2-TKIP"`, or `"OPEN"`
- `"weak_psk"`, `mode == "personal"` and `psk_length < 20`
- `"no_guest_isolation"`, `guest_network_isolated` is `False`
- `"weak_admin_password"`, `admin_password_strong` is `False`
- `"outdated_firmware"`, `firmware_current` is `False`

Examples

```
cfg = {
    "protocol": "WPA2-TKIP", "mode": "personal", "psk_length": 12,
    "guest_network_isolated": True, "admin_password_strong": True, "firmware_current": True,
}
evaluate_wifi_config(cfg)
# -> ["insecure_protocol", "weak_psk"]

cfg2 = {
    "protocol": "WPA3", "mode": "enterprise", "psk_length": 0,
    "guest_network_isolated": True, "admin_password_strong": True, "firmware_current": True,
}
evaluate_wifi_config(cfg2)
# -> []
```

Constraints

- Deterministic, offline, standard library only.
- `psk_length` is only checked when `mode == "personal"`; ignore it entirely for `"enterprise"` mode (enterprise mode uses per-user 802.1X credentials, not a shared PSK).
- Gap codes in the output list must be sorted alphabetically and contain no duplicates.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 3.2.a. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import Dict, List


def evaluate_wifi_config(cfg: Dict) -> List[str]:
    """Return the sorted list of security gap codes in a Wi-Fi AP configuration.

    Args:
        cfg: dict with keys 'protocol' (str), 'mode' (str: 'personal'/'enterprise'),
             'psk_length' (int), 'guest_network_isolated' (bool),
             'admin_password_strong' (bool), 'firmware_current' (bool).
    Returns:
        Sorted list (no duplicates) of gap codes; see prompt.md / interface.md for the exact
        codes and rules.
    """
    # TODO: implement the gap-detection rules per prompt.md
    raise NotImplementedError("implement evaluate_wifi_config() — see prompt.md")
```

3.2.b · Managerial controls and security policies

3 pts · auto-graded on the site

Coding problem, 3.2.b: Control-type classifier

Skill: Mitigate Risk · **Maps to:** 3.2.A, 3.2.B · **Language:** Python 3 (pytest)

Problem

You are building a control matrix tool for Crestfield Analytics' security program. Given a list of implemented security scenarios, classify each one by control family: `"managerial"`, `"technical"`, or `"physical"`, based on how the control is actually implemented (its `"mechanism"`), not on what risk it happens to address.

Function contract

Implement `classify_controls(scenarios)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `scenarios`: `list[dict]`, each with: - `"id"`: `str`, a scenario identifier - `"description"`: `str`, flavor text describing the scenario (not used for classification) - `"mechanism"`: `str`, one of `"policy_document"`, `"training_program"`, `"access_review_process"`, `"software_or_hardware_enforcement"`, `"physical_barrier"`

Output

A `list[str]` of classifications, **one per input scenario, in the same order as the input**. Each entry is one of `"managerial"`, `"technical"`, `"physical"`.

Examples

```
scenarios = [
    {"id": "s1", "description": "New hires sign a document listing allowed and prohibited use of company
laptops.",
     "mechanism": "policy_document"},
    {"id": "s2", "description": "The office network blocks known-malicious IP ranges automatically.",
     "mechanism": "software_or_hardware_enforcement"},
    {"id": "s3", "description": "The server room door requires a badge to open.",
     "mechanism": "physical_barrier"},
]
classify_controls(scenarios)  ->  ["managerial", "technical", "physical"]
```

Constraints

- Deterministic, offline, standard library only.
- Classification depends only on `"mechanism"`, never on `"description"` text.
- Mapping: `"policy_document"`, `"training_program"`, and `"access_review_process"` are all `"managerial"`; `"software_or_hardware_enforcement"` is `"technical"`; `"physical_barrier"` is `"physical"`.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 3.2.b. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import Dict, List


def classify_controls(scenarios: List[Dict]) -> List[str]:
    """Classify each scenario's control family by its 'mechanism' field.

    Args:
        scenarios: list of dicts with keys 'id' (str), 'description' (str),
                   'mechanism' (str: 'policy_document', 'training_program',
                   'access_review_process', 'software_or_hardware_enforcement',
                   'physical_barrier').
    Returns:
        List of classification strings, same length and order as `scenarios`, each one of
        "managerial", "technical", "physical".
        See prompt.md and interface.md for the exact rules.
    """
    # TODO: implement the classification rules per prompt.md
    raise NotImplementedError("implement classify_controls() — see prompt.md")
```

3.2.c · Layered network defense overview

3 pts · auto-graded on the site

Coding problem, 3.2.c: Defense-in-depth layer classifier

Skill: Mitigate Risk · **Maps to:** 3.2.B, 3.2.C · **Language:** Python 3 (pytest)

Problem

You are building a defense-in-depth mapping tool for Crestfield Analytics' security architecture review. Given a list of deployed controls, assign each one to its correct layer in the layered defense model taught in the lesson: `"perimeter"`, `"network"`, `"host"`, `"application"`, `"data"`, or `"human"`.

Function contract

Implement `classify_layer(controls)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `controls`: `list[dict]`, each with: - `"name"`: `str`, a human-readable control name (not used for classification) - `"category"`: `str`, one of the following exact values: - Perimeter: `"firewall"`, `"ips"`, `"dns_filter"` - Network: `"vlan_segmentation"`, `"internal_firewall"`, `"wifi_protocol"` - Host: `"endpoint_av"`, `"host_firewall"`, `"os_patching"` - Application: `"input_validation"`, `"waf"`, `"app_auth"` - Data: `"encryption_at_rest"`, `"dlp"`, `"data_classification"` - Human: `"awareness_training"`, `"aup"`, `"access_review"`, `"incident_response_policy"`

Output

A `list[str]` of layer assignments, **one per input control, in the same order as the input**. Each entry is one of `"perimeter"`, `"network"`, `"host"`, `"application"`, `"data"`, `"human"`.

Examples

```
controls = [
    {"name": "Border firewall", "category": "firewall"},
    {"name": "WPA3 wireless", "category": "wifi_protocol"},
    {"name": "Annual phishing training", "category": "awareness_training"},
]
classify_layer(controls)  ->  ["perimeter", "network", "human"]
```

Constraints

- Deterministic, offline, standard library only.
- Classification depends only on `"category"`, never on `"name"` text.
- `"category"` always contains one of the 16 listed values in test data (no invalid values).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 3.2.c. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import Dict, List


def classify_layer(controls: List[Dict]) -> List[str]:
    """Assign each control to its defense-in-depth layer based on 'category'.

    Args:
        controls: list of dicts with keys 'name' (str) and 'category' (str, one of 16 exact
                  values — see prompt.md / interface.md for the full mapping table).
```

```
Returns:
    List of layer strings, same length and order as `controls`, each one of
    "perimeter", "network", "host", "application", "data", "human".
"""
# TODO: implement the category -> layer mapping per prompt.md
raise NotImplementedError("implement classify_layer() — see prompt.md")
```